MANIT KHERA

One cardboard robot, three university units, and a student who could not leave it alone

0412 761 206 | manitkhera26@gmail.com | github.com/uiqvb | linkedin.com/in/manit-khera-603957252

About me

Hi, I'm Manit. Third-year Software Engineering student at Deakin, applying for the NXTGEN Intern Challenge from the software end, and apparently incapable of leaving a finished project alone.

Here is the pattern. I get an assignment. I finish it. Then I look at it and think: what else could this do? Last trimester that meant joining two completely unrelated assignments, a robot from one unit and a backend from another, purely because I wanted to watch my own code push a real thing across the floor. A month of build, break, rebuild later, the backend spoke and the robot listened.

I should credit my unit chair, Maksym. Every time I pitched an idea he would say "nah, you can't do that". Not to put me off: he had worked out that I do my best work trying to prove him wrong. Then, when I got there, he moved the target further out. He rage baited me through the whole unit, and in between we would sit there like two kids going "okay, but what if...". It was a proper love-hate arrangement and I would do it again.

Then the robot went to Open Day. Then into a third unit, where I asked what happens if there are 5,000 of it. So this is not really three projects. It is one robot that I keep volunteering for jobs it never applied for. I experiment, I break things on purpose, and I write down what happened, including the bits that still do not work.

Meet the robot

Build: two wheels, six sensors, one cardboard body.

Jobs so far: laundry attendant, class demo, Open Day exhibit, and the model for 5,000 imaginary colleagues.

Good at: running for shelter when it rains, and retracing its steps when it gets lost.

Bad at: wonky tape, and any battery below 90 percent.

Marks it earned me: 93 and 100/100.

it has never once complained about the extra work

"From Data to Decision to Action", on a cardboard budget

Things I built

I wired the robot and wrote its firmware, its Raspberry Pi dashboard and both backends myself. Chapters 1 to 3.

Data

Six cheap sensors and one unreliable rain sensor, which it cross-checks against a weather forecast before trusting. Chapter 1.

Decision

The server does all the thinking, and checks the robot's report in a simulation before believing it. Chapter 2.

Action

Commands to a real robot, which retraces its own steps when the signal goes quiet, at a scale of 5,000. Chapters 2 and 3.

Outside my lane

Software is home turf, but I wired it and chased its power fault myself. PCB, controls and LIDAR are what I want to learn next.

Chapter 1. The robot that saves your clothes

Smart Clothesline Robot. Embedded Systems unit, awarded 93.

Arduino Nano, C++, Raspberry Pi, Python/Flask, MQTT

Follow the build: 1 First demo → 2 Second demo → 3 Build write-up on Hackster.io → 4 Source code on GitHub

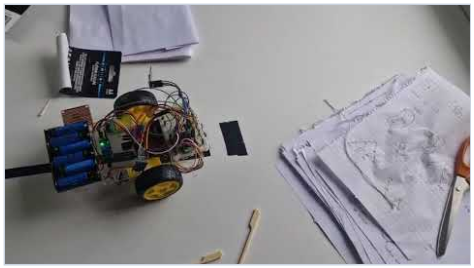
The idea. Clothes get rained on. So a small robot carries them between a sunny spot and a sheltered one, follows a line of black tape to get there, and turns to face the sun while it waits. I built and wired it myself: two wheels, three line sensors, two light sensors, one rain sensor and a cardboard body.

How I tested it. I got it wet. On purpose. Wetting the rain sensor sent it off to shelter, as it should. Then I lied to it: I fed it fake rain readings to check it would trust the weather forecast over a broken sensor. It did.

Plot twist. The robot had a voltage problem I could not see. On fresh batteries it behaved perfectly. Once they dropped below about 90 percent, it started acting up. So I kept buying new ones, probably $100 worth, before I tracked the real culprit, the voltage converter, down in the robot's own logs instead of with a multimeter.

I now own a large pile of batteries that are 90% full and useless to me

Still sulking. Its steering is simple, so it struggles on bad tape. Smarter steering is the next thing I want to try.



The robot on its tape track, from the first demo.

yes, the body really is cardboard

6
sensors, all wired by hand

93
unit mark

Chapter 2. Giving it a brain it never asked for

Robot Controller API. Backend Development unit, awarded 100/100.

.NET 8, C#, PostgreSQL, Arduino C++, Docker, Jenkins

Follow the build: 1 First demo, in class with my unit chair → 2 Open Day demo →

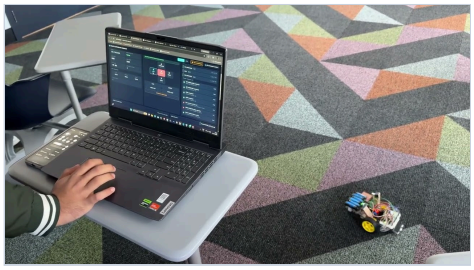
3 GitHub: first build, Open Day branch, deploying branch

The idea. A different unit entirely. I built a backend that does all the thinking, so a robot only has to ask "what next?" over Wi-Fi and do as it is told. Then I pointed it at the clothesline robot to see what would happen. For a while, not much: sometimes the backend worked and the robot ignored it, and sometimes the robot moved while the software believed something else entirely. Two confident liars.

My favourite what-if. What if the Wi-Fi drops? A robot with no plan of its own would be lost. So I built for it before it ever happened: every command arrives with its own undo. After 7 seconds of silence the robot retraces its last five moves by itself, back toward signal, then tells the server what it did. The server checks that story in a simulation before believing it. When I tested it on the real robot, it ended up exactly where the server predicted.

Party trick. It is also an RC car. You can drive it from the dashboard with the WASD keys, like a video game. There is nothing fancy underneath: no special radio link, only ordinary web requests. The robot keeps asking the server "anything for me?" and the server answers.

Where it went. A first demo in class with my unit chair, then selected for Deakin Open Day, where I ran it live for prospective students and their parents.



My dashboard driving the robot across the floor.

the robot is the small thing, bottom right

7 s	**100/100**
of silence before it rescues itself	unit mark

Chapter 3. What if there were 5,000 of them?

Scaling the Robot Dispatch API. Distributed Programming unit.

C#, ASP.NET Core, PostgreSQL, Azure, Docker, NBomber, xUnit

Follow the build: 1 Starting point: the Chapter 2 code → 2 Rebuilt code on GitHub → 3 Demo video → 4 Full report

The idea. One robot worked. So in a third unit I asked the uncomfortable question, built a pretend fleet in the cloud, and pointed it at my own backend. At 5,000 robots it completed zero jobs. Not few. Zero.

What I did about it. I went looking and found 12 problems, including one where two robots could grab the same job. I fixed them, taught the server to say "try again in a second" instead of falling over, added a second server, and ran it all again.

How I know it worked. 80 test runs, each setup repeated three times, plus 54 automated tests. I did not want one lucky run.

Still unknown. The new breaking point. My pretend fleet gave up before the backend did, which I am choosing to take as a compliment.

Jobs finished per second, 400 robots

Before	177
After, 1 server	608 (3.4×)
After, 2 servers	1,069 (6×)

Jobs finished per second, 5,000 robots

Before	0
After, 2 servers	1,008, none failed

from nothing at all to a thousand a second

Three things this robot taught me

A robot that says it moved has not necessarily moved. Check its story before you write it down.

Good logs beat a multimeter. If the robot records the right things, it will tell you what is wrong with it.

A perfect mark is not a stress test. 100/100 with one robot said nothing about 5,000.

What I have not done yet. Closed-loop control, PCB design, or working with real LIDAR. My line following is a state machine, not a feedback loop. Those are three of the areas on your list, and learning them alongside engineers on one shared problem is most of why I am applying. Where I can pull my weight from day one is the software and the code where the robot meets the backend. And the habit on this page comes with me: try it, break it on purpose, measure it, write it down, and bring it back for another round.